

## Contents

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>1 Basic</b>                                  | <b>1</b>  |
| 1.1 compile . . . . .                           | 1         |
| 1.2 default code . . . . .                      | 1         |
| 1.3 debug list . . . . .                        | 1         |
| 1.4 時間複雜度 . . . . .                             | 1         |
| <b>2 Dark Code</b>                              | <b>1</b>  |
| 2.1 IO optimization . . . . .                   | 1         |
| <b>3 Geometry</b>                               | <b>1</b>  |
| 3.1 2D point . . . . .                          | 1         |
| 3.2 兩線段交點 . . . . .                             | 2         |
| 3.3 兩圓交點 . . . . .                              | 2         |
| 3.4 Convex Hull . . . . .                       | 2         |
| <b>4 Flow</b>                                   | <b>2</b>  |
| 4.1 Dinic . . . . .                             | 2         |
| 4.2 min cost flow . . . . .                     | 3         |
| <b>5 Mathematics</b>                            | <b>3</b>  |
| 5.1 ax+by=gcd(a,b) . . . . .                    | 3         |
| 5.2 GaussElimination . . . . .                  | 3         |
| 5.3 Inverse . . . . .                           | 3         |
| 5.4 LinearPrime 歐拉篩 . . . . .                   | 4         |
| 5.5 Miller Rabin . . . . .                      | 4         |
| 5.6 Pollard's rho . . . . .                     | 4         |
| 5.7 NTT . . . . .                               | 4         |
| 5.8 數論基本工具 . . . . .                            | 4         |
| 5.9 Mobius . . . . .                            | 5         |
| 5.10SG . . . . .                                | 5         |
| 5.11歐拉定理 . . . . .                              | 5         |
| 5.12Theorem . . . . .                           | 5         |
| <b>6 Graph</b>                                  | <b>6</b>  |
| 6.1 BCC . . . . .                               | 6         |
| 6.2 Prim . . . . .                              | 6         |
| 6.3 Bellman Ford . . . . .                      | 6         |
| 6.4 Kruskal . . . . .                           | 6         |
| 6.5 Dijkstra . . . . .                          | 7         |
| 6.6 Strongly Connected Component(SCC) . . . . . | 7         |
| 6.7 Hungarian . . . . .                         | 7         |
| 6.8 KM . . . . .                                | 7         |
| 6.9 最小平均環 . . . . .                             | 8         |
| 6.10偵測負環 . . . . .                              | 8         |
| 6.11Tarjan . . . . .                            | 8         |
| 6.12Topological Sort . . . . .                  | 8         |
| <b>7 Data Structure</b>                         | <b>8</b>  |
| 7.1 2D Range Tree . . . . .                     | 8         |
| 7.2 Segment Tree . . . . .                      | 9         |
| 7.3 ZKW 線段樹 . . . . .                           | 9         |
| 7.4 Sparse Table . . . . .                      | 9         |
| 7.5 Lazy Tag . . . . .                          | 9         |
| 7.6 BIT 樹狀樹組 . . . . .                          | 10        |
| 7.7 並查集 union find . . . . .                    | 10        |
| 7.8 Treap . . . . .                             | 10        |
| 7.9 2D BIT . . . . .                            | 10        |
| <b>8 String</b>                                 | <b>11</b> |
| 8.1 KMP . . . . .                               | 11        |
| 8.2 smallest rotation . . . . .                 | 11        |
| 8.3 Suffix Array . . . . .                      | 11        |
| 8.4 Z-value . . . . .                           | 11        |
| 8.5 旋轉哈希 . . . . .                              | 12        |
| 8.6 後綴自動機 . . . . .                             | 12        |
| <b>9 Others</b>                                 | <b>12</b> |
| 9.1 矩陣樹定理 . . . . .                             | 12        |
| 9.2 1D/1D dp 優化 . . . . .                       | 12        |
| 9.3 Theorem - DP optimization . . . . .         | 13        |
| 9.4 Stable Marriage . . . . .                   | 13        |
| 9.5 莫隊 . . . . .                                | 13        |
| 9.6 莫隊帶修改 . . . . .                             | 13        |
| 9.7 矩陣乘法 . . . . .                              | 14        |
| 9.8 c++ 小抄 . . . . .                            | 14        |
| 9.9 python 小抄 . . . . .                         | 14        |
| 9.10萬年曆 . . . . .                               | 14        |
| 9.11多重背包 . . . . .                              | 14        |

## 1 Basic

### 1.1 compile

```
# preset before coding
echo "cd ~/Desktop" >> ~/.bashrc
gedit -> preference -> tab width: 4

# Editor
gedit a.cpp

# Compile
g++ a.cpp -std=c++14 -O2 -pipe -Wall

g++ -std=c++14 -Wall -Wextra -Wshadow -Wconversion
-g -fsanitize=address,undefined
main.cpp -o main
// -Wmisleading-indentation # 檢測縮排不對 for loop 沒
括號卻寫兩行
// -Wfatal-errors #讓編譯器只跳一個錯

ASAN_OPTIONS=detect_leaks=0 ./main #叫他不要叫mem leak
// -fsanitize=address 檢測記憶體違規存取

**All file will be compiled to a.out unless you use -o(
not recommended, just use a.out)**
# Run
./a.out

# Run with file input
./a.out < input.txt

# Run with file input and output
./a.out < input.txt > output.txt

# Python Run
python3 a.py < input.txt > output.txt

# Copy Paste In Ubuntu
* copy: ctrl+insert
* paste: shift+insert

# 比對文件相同
sdiff a.txt b.txt
```

### 1.2 default code

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}
```

### 1.3 debug list

記得測試 python 的內建函數庫有哪些  
bits/stdc++.h 跟 global variable y1 衝突，不能用  
模板要記得 init  
priority\_queue 要清空  
事先將把邊界測資加入測試  
邊界條件（過程溢位，題目數據範圍），會不會爆 long long  
是否讀錯題目，想不到時可以自己讀一次題目  
比較容易有問題的地方換人寫  
注意公式有沒有推錯或抄錯  
精度誤差 sqrt(太大的東西) + EPS  
喇分 random\_shuffle 隨機演算法

## 1.4 時間複雜度

| 時間複雜度           | 可處理的最大 $N$ 數量級 (約)                 |
|-----------------|------------------------------------|
| $O(1)$          | 幾乎沒限制                              |
| $O(\log N)$     | $10^{18}$ 級別 (如快速幂)                |
| $O(\sqrt{N})$   | $10^{10}$                          |
| $O(N)$          | $10^8$                             |
| $O(N \log N)$   | $10^6 \sim 5 \times 10^6$          |
| $O(N\sqrt{N})$  | $1 \times 10^5 \sim 2 \times 10^5$ |
| $O(N^2)$        | $3 \times 10^3 \sim 10^4$          |
| $O(N^2 \log N)$ | $10^3 \sim 3 \times 10^3$          |
| $O(N^3)$        | $300 \sim 500$                     |
| $O(2^N)$        | $N \leq 20$                        |
| $O(N!)$         | $N \leq 10$                        |

## 2 Dark Code

### 2.1 IO optimization

```
// fread input. read(x) returns false on eof.
struct fast_input {
    static constexpr size_t size = 1 << 16;
    array<char, size> buf{};
    size_t pos = 0, len = 0;

    int get() {
        if (pos == len) {
            len = fread(buf.data(), 1, size, stdin);
            pos = 0;
            if (!len) return EOF;
        }
        return buf[pos++];
    }

    template<class t>
    bool read(t &x) {
        int c = get(), sign = 1;
        while (c != EOF && c != '-' && (c < '0' || c > '9')) c = get();
        if (c == EOF) return false;
        if (c == '-') sign = -1, c = get();
        x = 0;
        while ('0' <= c && c <= '9') x = x * 10 + c - '0', c = get();
        x *= sign;
        return true;
    }
};
```

## 3 Geometry

### 3.1 2D point

```
using ld = long double;
const ld eps = 1e-12L;

struct point {
    ld x = 0, y = 0;
    point() = default;
    point(ld x, ld y) : x(x), y(y) {}
    point operator+(point p) const { return {x + p.x, y + p.y}; }
    point operator-(point p) const { return {x - p.x, y - p.y}; }
    point operator*(ld k) const { return {x * k, y * k}; }
    bool operator<(point p) const { return tie(x, y) < tie(p.x, p.y); }
};

ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }
ld norm2(point p) { return dot(p, p); }
ld norm(point p) { return sqrtl(norm2(p)); }
```

### 3.2 兩線段交點

```
using i128 = __int128_t;

struct ipoint {
    long long x, y;
};

i128 cross(ipoint a, ipoint b, ipoint c) {
    return (i128(b.x) - a.x) * (i128(c.y) - a.y) -
           (i128(b.y) - a.y) * (i128(c.x) - a.x);
}

int sign(i128 x) { return (x > 0) - (x < 0); }

bool on_segment(ipoint a, ipoint b, ipoint p) {
    return cross(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segments_intersect(ipoint a, ipoint b, ipoint c, ipoint d) {
    i128 x = cross(a, b, c), y = cross(a, b, d);
    i128 z = cross(c, d, a), w = cross(c, d, b);
    if (!x && on_segment(a, b, c)) return true;
    if (!y && on_segment(a, b, d)) return true;
    if (!z && on_segment(c, d, a)) return true;
    if (!w && on_segment(c, d, b)) return true;
    return sign(x) != sign(y) && sign(z) != sign(w);
}
```

### 3.3 兩圓交點

```
// count: -1 identical circles; otherwise number of points
struct circle_hit {
    int count;
    vector<point> p;
};

circle_hit circle_intersections(point a, ld r, point b, ld s) {
    point d = b - a;
    ld d2 = norm2(d), len = sqrtl(d2);
    if (len <= eps && fabs1(r - s) <= eps) return {-1, {}};
    if (len <= eps || len > r + s + eps || len < fabs1(r - s) - eps)
        return {0, {}};

    ld x = (d2 + r * r - s * s) / (2 * len);
    ld h2 = max<ld>(0, r * r - x * x);
    point mid = a + d * (x / len);
    if (h2 <= eps) return {1, {mid}};
    point off{-d.y * sqrtl(h2) / len, d.x * sqrtl(h2) / len};
    return {2, {mid + off, mid - off}};
}
```

### 3.4 Convex Hull

```
// needs point, cross and eps from 2Dpoint.cpp
vector<point> convex_hull(vector<point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end(), [](point a, point b) {
        return fabs1(a.x - b.x) <= eps && fabs1(a.y - b.y) <= eps;
    }), p.end());
    if (p.size() <= 1) return p;

    vector<point> h;
    for (int pass = 0; pass < 2; ++pass) {
        size_t base = h.size();
```

```

    for (point q : p) {
        while (h.size() >= base + 2 &&
            cross(h.back() - h[h.size() - 2], q
                - h.back()) <= eps)
            h.pop_back();
        h.push_back(q);
    }
    h.pop_back();
    reverse(p.begin(), p.end());
}
return h; // ccw; first point is not repeated
}

```

## 4 Flow

### 4.1 Dinic

```

using ll = long long;
const ll inf = LLONG_MAX / 4;

struct dinic {
    struct edge { int to, rev; ll cap; };
    vector<vector<edge>> g;
    vector<int> level, it;

    dinic(int n) : g(n), level(n), it(n) {}
    void add_edge(int u, int v, ll cap) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, cap});
        g[v].push_back({u, a, 0});
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u]) if (e.cap && level[e.to] == -1)
                level[e.to] = level[u] + 1, q.push(e.to);
        }
        return level[t] != -1;
    }
    ll dfs(int u, int t, ll flow) {
        if (u == t) return flow;
        for (int &i = it[u]; i < (int)g[u].size(); ++i)
            if (!e.cap || level[e.to] != level[u] + 1) continue;
            ll got = dfs(e.to, t, min(flow, e.cap));
            if (!got) continue;
            e.cap -= got;
            g[e.to][e.rev].cap += got;
            return got;
        }
        return 0;
    }
    ll max_flow(int s, int t) {
        ll ans = 0, got;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while ((got = dfs(s, t, inf))) ans += got;
        }
        return ans;
    }
    vector<char> min_cut_side(int s) const {
        vector<char> seen(g.size());
        vector<int> stack{ s };
        seen[s] = true;
        while (!stack.empty()) {
            int u = stack.back();
            stack.pop_back();
            for (auto &e : g[u]) if (e.cap && !seen[e.to])

```

```

                seen[e.to] = true, stack.push_back(e.to);
            }
        }
        return seen;
    }
};

/*
Lower bounds [low, high]:
add u->v with high-low; balance[u]-=low, balance[v]+=low.
add ss->v for positive balance, v->tt for negative balance.
for s-t flow also add t->s with inf before the feasibility flow.
after feasibility, remove ss/tt and t->s:
max = base + max_flow(s,t), min = base - max_flow(t,s).
*/

```

### 4.2 min cost flow

```

using ll = long long;
const ll inf = LLONG_MAX / 4;

struct min_cost_flow {
    struct edge { int to, rev; ll cap, cost; };
    vector<vector<edge>> g;

    min_cost_flow(int n) : g(n) {}
    void add_edge(int u, int v, ll cap, ll cost) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, cap, cost});
        g[v].push_back({u, a, 0, -cost});
    }
    pair<ll, ll> solve(int s, int t, ll limit = inf) {
        ll flow = 0, cost = 0;
        int n = g.size();
        vector<ll> dist(n);
        vector<int> pv(n), pe(n);
        vector<char> in_queue(n);
        while (flow < limit) {
            fill(dist.begin(), dist.end(), inf);
            fill(in_queue.begin(), in_queue.end(), false);
            queue<int> q;
            dist[s] = 0;
            q.push(s);
            in_queue[s] = true;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                in_queue[u] = false;
                for (int i = 0; i < (int)g[u].size(); ++i) {
                    edge &e = g[u][i];
                    __int128 raw = (__int128)dist[u] + e.cost;
                    ll nd = raw < -inf ? -inf : raw > inf ? inf : (ll)raw;
                    if (e.cap && dist[e.to] > nd) {
                        dist[e.to] = nd;
                        pv[e.to] = u, pe[e.to] = i;
                        if (!in_queue[e.to]) q.push(e.to), in_queue[e.to] = true;
                    }
                }
            }
            if (dist[t] == inf) break;
            ll add = limit - flow;
            for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
            for (int v = t; v != s; v = pv[v]) {
                edge &e = g[pv[v]][pe[v]];
                e.cap -= add;
                g[v][e.rev].cap += add;
            }
            flow += add;
            cost += add * dist[t];
        }
        return {flow, cost};
    }
};

```

```
    }
};
```

## 5 Mathematics

### 5.1 $ax+by=gcd(a,b)$

```
using ll = long long;

// returns {g, x, y}: ax + by = g = gcd(a,b)
tuple<ll, ll, ll> ext_gcd(ll a, ll b) {
    if (!b) return {abs(a), a < 0 ? -1 : 1, 0};
    ll g, x, y;
    tie(g, x, y) = ext_gcd(b, a % b);
    return {g, y, x - a / b * y};
}
```

### 5.2 GaussElimination

```
using ld = long double;
const ld eps = 1e-12L;

// augmented matrix; returns 0=no solution, 1=unique,
// 2=infinite
int gauss(vector<vector<ld>> a, vector<ld> &answer) {
    int n = a.size(), m = a.empty() ? 0 : a[0].size() - 1, row = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && row < n; ++col) {
        int best = row;
        for (int i = row; i < n; ++i)
            if (fabsl(a[i][col]) > fabsl(a[best][col]))
                best = i;
        if (fabsl(a[best][col]) <= eps) continue;
        swap(a[best], a[row]);
        where[col] = row;
        ld div = a[row][col];
        for (int j = col; j <= m; ++j) a[row][j] /= div;
        for (int i = 0; i < n; ++i) if (i != row) {
            ld mul = a[i][col];
            for (int j = col; j <= m; ++j) a[i][j] -= mul * a[row][j];
        }
        ++row;
    }
    answer.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] != -1)
        answer[i] = a[where[i]][m];
    for (auto &r : a) {
        ld sum = 0;
        for (int i = 0; i < m; ++i) sum += r[i] * answer[i];
        if (fabsl(sum - r[m]) > eps) return 0;
    }
    return count(where.begin(), where.end(), -1) ? 2 : 1;
}
```

### 5.3 Inverse

```
using ll = long long;

ll power(ll a, ll n, ll mod) {
    ll ans = 1;
    for (; n; n /= 2, a = (__int128)a * a % mod)
        if (n & 1) ans = (__int128)ans * a % mod;
    return ans;
}

// mod must be prime; 1 <= n < mod
vector<ll> inverse_table(int n, ll mod) {
    assert(1 <= n && n < mod);
    vector<ll> inv(n + 1);
```

```
    inv[1] = 1;
    for (int i = 2; i <= n; ++i)
        inv[i] = mod - (__int128)(mod / i) * inv[mod % i] % mod;
    return inv;
}
```

### 5.4 LinearPrime 歐拉篩

```
vector<int> linear_sieve(int n) {
    vector<int> primes, min_prime(n + 1);
    for (int i = 2; i <= n; ++i) {
        if (!min_prime[i]) min_prime[i] = i, primes.push_back(i);
        for (int p : primes) {
            if (p > min_prime[i] || (long long)i * p > n) break;
            min_prime[i * p] = p;
        }
    }
    return primes;
}
```

### 5.5 Miller Rabin

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mod_mul(u64 a, u64 b, u64 mod) { return u128(a) * b % mod; }
u64 mod_power(u64 a, u64 n, u64 mod) {
    u64 ans = 1;
    for (; n; n /= 2, a = mod_mul(a, a, mod))
        if (n & 1) ans = mod_mul(ans, a, mod);
    return ans;
}

bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : array<u64, 7>{{2, 3, 5, 7, 11, 13, 17}})
        if (n % p == 0) return n == p;
    u64 d = n - 1, s = 0;
    while (!(d & 1)) d /= 2, ++s;
    for (u64 a : array<u64, 7>{{2, 3, 325, 9375, 28178, 450775, 9780504, 1795265022}}) {
        if (a % n == 0) continue;
        u64 x = mod_power(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (u64 r = 1; r < s; ++r) {
            x = mod_mul(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}
```

### 5.6 Pollard's rho

```
// needs is_prime() and mod_mul() from Miller-Rabin.cpp
using u64 = uint64_t;

u64 pollard(u64 n) {
    if (!(n & 1)) return 2;
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (true) {
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 x = uniform_int_distribution<u64>(0, n - 1)(rng), y = x, d = 1;
```

```

    auto f = [&](u64 v) { return ((__uint128_t(
        mod_mul(v, v, n)) + c) % n; };
    while (d == 1) {
        x = f(x), y = f(f(y));
        d = __gcd(x > y ? x - y : y - x, n);
    }
    if (d != n) return d;
}

}

void factor(u64 n, map<u64, int> &count) {
    if (n == 1) return;
    if (is_prime(n)) return ++count[n], void();
    u64 d = pollard(n);
    factor(d, count);
    factor(n / d, count);
}

```

## 5.7 NTT

```

// coefficients must be in [0, mod)
const int mod = 998244353, root = 3;

int power(int a, int n) {
    long long ans = 1;
    for (; n; n /= 2, a = (long long)a * a % mod)
        if (n & 1) ans = ans * a % mod;
    return ans;
}

void ntt(vector<int> &a, bool inverse) {
    int n = a.size();
    assert(n && !(n & (n - 1)) && (mod - 1) % n == 0);
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n / 2;
        for (; j & bit; bit /= 2) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len *= 2) {
        int wlen = power(root, (mod - 1) / len);
        if (inverse) wlen = power(wlen, mod - 2);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j], v = w * a[i + j + len / 2] % mod;
                a[i + j] = u + v < mod ? u + v : u + v - mod;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + mod;
                w = w * wlen % mod;
            }
        }
    }
    if (inverse) {
        int inv = power(n, mod - 2);
        for (int &x : a) x = (long long)x * inv % mod;
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n *= 2;
    assert(n <= (1 << 23));
    a.resize(n), b.resize(n);
    ntt(a, false), ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (long long)a[i] * b[i] % mod;
    ntt(a, true);
    a.resize(need);
    return a;
}

```

## 5.8 數論基本工具

```

using ll = long long;

ll mod_power(ll a, ll n, ll mod) {
    ll ans = 1;
    for (; n; n /= 2, a = (__int128)a * a % mod)
        if (n & 1) ans = (__int128)ans * a % mod;
    return ans;
}

// mod must be prime and n < mod
struct combinations {
    ll mod;
    vector<ll> fact, inv_fact;
    combinations(int n, ll mod) : mod(mod), fact(n + 1, 1), inv_fact(n + 1) {
        for (int i = 1; i <= n; ++i) fact[i] = fact[i - 1] * i % mod;
        inv_fact[n] = mod_power(fact[n], mod - 2, mod);
        for (int i = n; i; --i) inv_fact[i - 1] = inv_fact[i] * i % mod;
    }
    ll choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return fact[n] * inv_fact[k] % mod * inv_fact[n - k] % mod;
    }
};

```

## 5.9 Mobius

```

vector<int> mobius(int n) {
    vector<int> mu(n + 1), primes;
    vector<char> composite(n + 1);
    if (n < 1) return mu;
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!composite[i]) primes.push_back(i), mu[i] = -1;
        for (int p : primes) {
            if ((long long)i * p > n) break;
            composite[i * p] = true;
            if (i % p == 0) { mu[i * p] = 0; break; }
            mu[i * p] = -mu[i];
        }
    }
    return mu;
}

```

## 5.10 SG

```

/*
finite impartial game:
sg[u] = mex({sg[v] | u -> v}).
xor the sg values only when combining independent games
.
normal play: xor != 0 wins.
misere nim: if every pile is 1, odd pile count loses;
otherwise xor != 0 wins.
*/
vector<int> Grundy(const vector<vector<int>> &g) {
    vector<int> sg(g.size(), -1);
    function<int(int)> dfs = [&](int u) {
        if (sg[u] != -1) return sg[u];
        vector<int> seen(g[u].size() + 1);
        for (int v : g[u]) {
            int x = dfs(v);
            if (x < (int)seen.size()) seen[x] = true;
        }
        return sg[u] = find(seen.begin(), seen.end(), 0) - seen.begin();
    };
    for (int u = 0; u < (int)g.size(); ++u) dfs(u);
    return sg;
}

```

## 5.11 歐拉定理

歐拉降冪對於任何正整數  $m$ 、整數  $a$  和非負整數  $k$

$$a^k \equiv \begin{cases} a^{k \bmod \varphi(m)}, & \gcd(a, m) = 1, \\ a^k, & \gcd(a, m) \neq 1, \quad k < \varphi(m), \quad (\bmod m) \\ a^{(k \bmod \varphi(m)) + \varphi(m)}, & \gcd(a, m) \neq 1, \quad k \geq \varphi(m). \end{cases}$$

## 5.12 Theorem

```
/*
Lucas's Theorem
For non-negative integer n,m and prime P,
C(m,n) mod P = C(m/P,n/P) * C(m%P,n%P) mod P
= mult_i ( C(m_i,n_i) )
where m_i is the i-th digit of m in base P.
-----
Pick's Theorem
A = i + b/2 - 1
-----
Kirchhoff's theorem
A_{ii} = deg(i), A_{ij} = (i,j) \in E ? -1 : 0
Deleting any one row, one column, and cal the det(A)
-----
Nth Catalan recursive function:
C_0 = 1, C_{n+1} = C_n * 2(2n + 1)/(n+2)
-----
Mobius Formula
u(n) = 1, if n = 1
      (-1)^m, 若 n 無平方數因數, 且 n = p1*p2*p3*...*pk
      0, 若 n 有大於 1 的平方數因數
- Property
1. (積性函數) u(a)u(b) = u(ab)
2. \sum_{d|n} u(d) = [n == 1]
-----
Mobius Inversion Formula
if f(n) = \sum_{d|n} g(d)
then g(n) = \sum_{d|n} u(n/d)f(d)
      = \sum_{d|n} u(d)f(n/d)
- Application
the number/power of gcd(i, j) = k
- Trick
分塊, O(sqrt(n))
-----
Chinese Remainder Theorem (m_i 兩兩互質)

x = a_1 (mod m_1)
x = a_2 (mod m_2)
....
x = a_i (mod m_i)

construct a solution:

Let M = m_1 * m_2 * m_3 * ... * m_n
Let M_i = M / m_i

t_i = 1 / M_i
t_i * M_i = 1 (mod m_i)

solution x = a_1 * t_1 * M_1 + a_2 * t_2 * M_2 + ...
            + a_n * t_n * M_n + k * M
            = k*M + \sum a_i * t_i * M_i, k is any integer.

under mod M, there is one solution x = \sum a_i * t_i * M_i
-----
Burnside's Lemma
|G| * |X/G| = sum( |X^g| ) where g in G
總方法數: 每一種旋轉下不動點的個數總和 除以 旋轉的方法數
*/
```

## 6 Graph

### 6.1 BCC

```
// undirected edge-biconnected components; parallel
// edges are allowed
struct edge_bcc {
    struct edge { int to, id; };
    vector<vector<edge>> g;
    vector<int> dfn, low, comp;
    vector<char> bridge;
    int time = 0;

    edge_bcc(int n) : g(n), dfn(n), low(n), comp(n, -1) {}
    void add_edge(int u, int v) {
        int id = bridge.size();
        bridge.push_back(false);
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    void dfs(int u, int parent_edge = -1) {
        dfn[u] = low[u] = ++time;
        for (edge e : g[u]) {
            if (e.id == parent_edge) continue;
            if (!dfn[e.to]) {
                dfs(e.to, e.id);
                low[u] = min(low[u], low[e.to]);
                if (low[e.to] > dfn[u]) bridge[e.id] = true;
            } else low[u] = min(low[u], dfn[e.to]);
        }
    }
    void paint(int u, int id) {
        comp[u] = id;
        for (edge e : g[u])
            if (!bridge[e.id] && comp[e.to] == -1)
                paint(e.to, id);
    }
    int solve() {
        for (int u = 0; u < (int)g.size(); ++u) if (!dfn[u]) dfs(u);
        int count = 0;
        for (int u = 0; u < (int)g.size(); ++u)
            if (comp[u] == -1) paint(u, count++);
        return count;
    }
};
```

### 6.2 Prim

```
using ll = long long;
using edge = pair<ll, int>; // weight, to

// {connected, mst weight}; zero and parallel edges are
// allowed
pair<bool, ll> prim(const vector<vector<edge>> &g, int
root = 0) {
    if (g.empty()) return {true, 0};
    vector<char> used(g.size());
    priority_queue<edge, vector<edge>, greater<edge>>
        pq;
    pq.push({0, root});
    ll total = 0;
    int seen = 0;
    while (!pq.empty()) {
        ll w = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (used[u]) continue;
        used[u] = true;
        total += w;
        ++seen;
        for (edge e : g[u]) if (!used[e.second]) pq.
            push(e);
    }
    return {seen == (int)g.size(), total};
}
```

### 6.3 Bellman Ford

```
using ll = long long;
const ll inf = LLONG_MAX / 4;

struct edge {
    int from, to;
    ll weight;
};

struct bellman_result {
    vector<ll> dist;
    vector<int> parent;
    bool negative_cycle;
};

bellman_result bellman_ford(int n, int source, const
    vector<edge> &edges) {
    vector<ll> dist(n, inf);
    vector<int> parent(n, -1);
    dist[source] = 0;
    bool changed = false;
    for (int i = 0; i < n; ++i) {
        changed = false;
        for (edge e : edges) {
            if (dist[e.from] == inf) continue;
            __int128 raw = (__int128)dist[e.from] + e.
                weight;
            ll nd = raw < -inf ? -inf : raw > inf ? inf
                : (ll)raw;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                parent[e.to] = e.from;
                changed = true;
            }
        }
        if (!changed) break;
    }
    return {dist, parent, changed}; // reachable
    negative cycle iff changed
}
```

### 6.4 Kruskal

```
using ll = long long;

struct edge {
    int u, v;
    ll weight;
};

struct dsu {
    vector<int> p, size;
    dsu(int n) : p(n), size(n, 1) { iota(p.begin(), p.
        end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] =
        find(p[x]); }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        p[b] = a;
        size[a] += size[b];
        return true;
    }
};

pair<bool, ll> kruskal(int n, vector<edge> edges) {
    sort(edges.begin(), edges.end(), [](edge a, edge b)
        {
            return a.weight < b.weight;
        });
    dsu uf(n);
    ll total = 0;
    int used = 0;
    for (edge e : edges) if (uf.unite(e.u, e.v))
        total += e.weight, ++used;
    return {used == n - 1, total};
}
```

### 6.5 Dijkstra

```
using ll = long long;
using graph_edge = pair<int, ll>; // to, weight
using state = pair<ll, int>; // distance, vertex
const ll inf = LLONG_MAX / 4;

vector<ll> dijkstra(int source, const vector<vector<
    graph_edge>> &g) {
    vector<ll> dist(g.size(), inf);
    priority_queue<state, vector<state>, greater<state>
        >> pq;
    dist[source] = 0;
    pq.push({0, source}); // distance, vertex
    while (!pq.empty()) {
        ll d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d != dist[u]) continue;
        for (graph_edge e : g[u]) {
            int v = e.first;
            ll w = e.second;
            assert(w >= 0);
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

### 6.6 Strongly Connected Component(SCC)

```
struct kosaraju {
    vector<vector<int>> g, rev;
    vector<int> used, order, comp;

    kosaraju(int n) : g(n), rev(n), used(n), comp(n) {}
    void add_edge(int u, int v) { g[u].push_back(v),
        rev[v].push_back(u); }
    void dfs(int u) {
        used[u] = true;
        for (int v : g[u]) if (!used[v]) dfs(v);
        order.push_back(u);
    }
    void rdfs(int u, int id) {
        comp[u] = id;
        used[u] = true;
        for (int v : rev[u]) if (!used[v]) rdfs(v, id);
    }
    int solve() {
        fill(used.begin(), used.end(), 0);
        order.clear();
        for (int u = 0; u < (int)g.size(); ++u) if (!
            used[u]) dfs(u);
        fill(used.begin(), used.end(), 0);
        reverse(order.begin(), order.end());
        int count = 0;
        for (int u : order) if (!used[u]) rdfs(u, count
            ++);
        return count;
    }
};
```

### 6.7 Hungarian

```
// maximum cardinality bipartite matching
struct bipartite_matching {
    vector<vector<int>> g;
    vector<int> right_match, seen;
    int stamp = 0;

    bipartite_matching(int left, int right)
        : g(left), right_match(right, -1), seen(right)
        {}
}
```

```

void add_edge(int left, int right) { g[left].
    push_back(right); }
bool dfs(int u) {
    for (int v : g[u]) if (seen[v] != stamp) {
        seen[v] = stamp;
        if (right_match[v] == -1 || dfs(right_match
            [v]))
            return right_match[v] = u, true;
    }
    return false;
}
int solve() {
    int ans = 0;
    for (int u = 0; u < (int)g.size(); ++u) ++stamp
        , ans += dfs(u);
    return ans;
}
};

```

## 6.8 KM

```

using ll = long long;
const ll missing = LLONG_MIN / 4;
const ll inf = LLONG_MAX / 4;

struct assignment_result {
    bool perfect;
    ll weight;
    vector<int> right_match;
};

// square maximum-weight assignment; use missing for
// absent edges
assignment_result max_weight_matching(const vector<
    vector<ll>> &weight) {
    int n = weight.size();
    for (auto &row : weight) assert((int)row.size() ==
        n);
    vector<ll> u(n + 1), v(n + 1);
    vector<int> p(n + 1), way(n + 1);

    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        vector<ll> best(n + 1, inf);
        vector<char> used(n + 1);
        int col = 0;
        do {
            used[col] = true;
            int row = p[col], next = 0;
            ll delta = inf;
            for (int j = 1; j <= n; ++j) if (!used[j])
                {
                    ll cost = weight[row - 1][j - 1] ==
                        missing
                        ? inf / 2 : -weight[row - 1][j
                            - 1];
                    ll cur = cost - u[row] - v[j];
                    if (cur < best[j]) best[j] = cur, way[j]
                        = col;
                    if (best[j] < delta) delta = best[j],
                        next = j;
                }
            for (int j = 0; j <= n; ++j)
                if (used[j]) u[p[j]] += delta, v[j] -=
                    delta;
            else best[j] -= delta;
            col = next;
        } while (p[col]);
        do {
            int prev = way[col];
            p[col] = p[prev];
            col = prev;
        } while (col);
    }

    vector<int> match(n, -1);
    ll sum = 0;
    for (int col = 1; col <= n; ++col) {
        int row = p[col] - 1;
        match[col - 1] = row;
    }
}

```

```

if (weight[row][col - 1] == missing) return {
    false, 0, {}};
sum += weight[row][col - 1];
}
return {true, sum, match};
}

```

## 6.9 最小平均環

```

using ld = long double;
const ld inf = 1e100L;

struct edge {
    int from, to;
    ld weight;
};

// {has cycle, minimum mean}; karp, O(nm)
pair<bool, ld> min_mean_cycle(int n, const vector<edge>
    &edges) {
    vector<vector<ld>> dp(n + 1, vector<ld>(n, inf));
    fill(dp[0].begin(), dp[0].end(), 0); // super
    source
    for (int k = 1; k <= n; ++k)
        for (edge e : edges)
            if (dp[k - 1][e.from] != inf)
                dp[k][e.to] = min(dp[k][e.to], dp[k -
                    1][e.from] + e.weight);

    ld ans = inf;
    for (int v = 0; v < n; ++v) if (dp[n][v] != inf) {
        ld worst = -inf;
        for (int k = 0; k < n; ++k) if (dp[k][v] != inf
            )
            worst = max(worst, (dp[n][v] - dp[k][v]) /
                (n - k));
        ans = min(ans, worst);
    }
    return {ans != inf, ans};
}

```

## 6.10 偵測負環

```

using ll = long long;
const ll inf = LLONG_MAX / 4;

struct floyd_result {
    vector<vector<ll>> dist;
    vector<vector<char>> negative;
};

floyd_result floyd_warshall(vector<vector<ll>> dist) {
    int n = dist.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i) if (dist[i][k] !=
            inf)
            for (int j = 0; j < n; ++j) if (dist[k][j]
                != inf) {
                __int128 x = (__int128)dist[i][k] +
                    dist[k][j];
                ll value = x < -inf ? -inf : x > inf ?
                    inf : (ll)x;
                dist[i][j] = min(dist[i][j], value);
            }

    vector<vector<char>> negative(n, vector<char>(n));
    for (int k = 0; k < n; ++k) if (dist[k][k] < 0)
        for (int i = 0; i < n; ++i) if (dist[i][k] !=
            inf)
            for (int j = 0; j < n; ++j) if (dist[k][j]
                != inf)
                negative[i][j] = true;
    return {dist, negative};
}

```

## 6.11 Tarjan

```
// directed strongly connected components
struct tarjan {
    vector<vector<int>> g;
    vector<int> dfn, low, comp, stack;
    vector<char> in_stack;
    int time = 0, count = 0;

    tarjan(int n)
        : g(n), dfn(n), low(n), comp(n, -1), in_stack(n) {}

    void add_edge(int u, int v) { g[u].push_back(v); }
    void dfs(int u) {
        dfn[u] = low[u] = ++time;
        stack.push_back(u);
        in_stack[u] = true;
        for (int v : g[u]) {
            if (!dfn[v]) dfs(v), low[u] = min(low[u], low[v]);
            else if (in_stack[v]) low[u] = min(low[u], dfn[v]);
        }
        if (low[u] != dfn[u]) return;
        while (true) {
            int v = stack.back();
            stack.pop_back();
            in_stack[v] = false;
            comp[v] = count;
            if (v == u) break;
        }
        ++count;
    }

    int solve() {
        for (int u = 0; u < (int)g.size(); ++u) if (!dfn[u]) dfs(u);
        return count;
    }
};
```

## 6.12 Topological Sort

```
// empty result means the graph has a cycle
vector<int> topological_sort(const vector<vector<int>>& g) {
    vector<int> indeg(g.size()), order;
    for (auto &es : g) for (int v : es) ++indeg[v];
    queue<int> q;
    for (int i = 0; i < (int)g.size(); ++i)
        if (!indeg[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : g[u]) if (--indeg[v]) q.push(v);
    }
    if (order.size() != g.size()) order.clear();
    return order;
}
```

## 7 Data Structure

### 7.1 2D Range Tree

```
// static rectangle point count, inclusive bounds
struct range_tree_2d {
    vector<int> xs;
    vector<vector<int>> ys;

    range_tree_2d(vector<pair<int, int>> p) {
        sort(p.begin(), p.end());
        for (auto q : p) xs.push_back(q.first);
        ys.resize(4 * max<size_t>(1, p.size()));
        if (!p.empty()) build(1, 0, p.size(), p);
    }
};
```

```
void build(int o, int l, int r, const vector<pair<int, int>> &p) {
    if (r - l == 1) return ys[o].push_back(p[l].second);
    int m = (l + r) / 2;
    build(o * 2, l, m, p);
    build(o * 2 + 1, m, r, p);
    merge(ys[o * 2].begin(), ys[o * 2].end(),
          ys[o * 2 + 1].begin(), ys[o * 2 + 1].end(),
          back_inserter(ys[o]));
}

int query(int ql, int qr, int y1, int y2, int o, int l, int r) const {
    if (ql <= l && r <= qr)
        return upper_bound(ys[o].begin(), ys[o].end(), y2) -
               lower_bound(ys[o].begin(), ys[o].end(), y1);
    int m = (l + r) / 2, ans = 0;
    if (ql < m) ans += query(ql, qr, y1, y2, o * 2, l, m);
    if (m < qr) ans += query(ql, qr, y1, y2, o * 2 + 1, m, r);
    return ans;
}

int count(int x1, int y1, int x2, int y2) const {
    int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
    int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin();
    return l == r ? 0 : query(l, r, y1, y2, 1, 0, xs.size());
}
};
```

### 7.2 Segment Tree

```
// range add, range maximum; public ranges are [l, r]
struct segtree {
    struct node { long long mx = 0, lazy = 0; };
    int n;
    vector<node> t;

    segtree(int n) : n(n), t(4 * max(1, n)) { assert(n > 0); }
    void apply(int o, long long v) { t[o].mx += v, t[o].lazy += v; }
    void push(int o) {
        if (!t[o].lazy) return;
        apply(o * 2, t[o].lazy);
        apply(o * 2 + 1, t[o].lazy);
        t[o].lazy = 0;
    }
    void add(int ql, int qr, long long v, int o, int l, int r) {
        if (ql <= l && r <= qr) return apply(o, v);
        push(o);
        int m = (l + r) / 2;
        if (ql < m) add(ql, qr, v, o * 2, l, m);
        if (m < qr) add(ql, qr, v, o * 2 + 1, m, r);
        t[o].mx = max(t[o * 2].mx, t[o * 2 + 1].mx);
    }
    long long max_query(int ql, int qr, int o, int l, int r) {
        if (ql <= l && r <= qr) return t[o].mx;
        push(o);
        int m = (l + r) / 2;
        long long ans = LLONG_MIN;
        if (ql < m) ans = max(ans, max_query(ql, qr, o * 2, l, m));
        if (m < qr) ans = max(ans, max_query(ql, qr, o * 2 + 1, m, r));
        return ans;
    }
    void add(int l, int r, long long v) {
        if (l <= r) add(l, r + 1, v, 1, 0, n);
    }
    long long max_query(int l, int r) {
        assert(l <= r);
    }
};
```

```

    return max_query(l, r + 1, 1, 0, n);
}
};

```

### 7.3 ZKW 線段樹

```

// iterative segment tree: point add and range sum [l, r]
struct zkw_sum {
    int n = 1;
    vector<long long> t;
    zkw_sum(int size) {
        while (n < size) n *= 2;
        t.assign(2 * n, 0);
    }
    void add(int p, long long v) {
        for (t[p += n] += v, p /= 2; p; p /= 2)
            t[p] = t[p * 2] + t[p * 2 + 1];
    }
    long long sum(int l, int r) const {
        long long ans = 0;
        for (l += n, r += n + 1; l < r; l /= 2, r /= 2)
            {
                if (l & 1) ans += t[l++];
                if (r & 1) ans += t[--r];
            }
        return ans;
    }
};

```

### 7.4 Sparse Table

```

// idempotent rmq: minimum on [l, r]
template<class t>
struct sparse_table {
    vector<int> lg;
    vector<vector<t>> st;

    sparse_table(const vector<t> &a) : lg(a.size() + 1)
    {
        for (int i = 2; i < (int)lg.size(); ++i) lg[i]
            = lg[i / 2] + 1;
        int k = a.empty() ? 0 : lg[a.size()] + 1;
        st.assign(k, vector<t>(a.size()));
        if (a.empty()) return;
        st[0] = a;
        for (int j = 1; j < k; ++j)
            for (int i = 0; i + (1 << j) <= (int)a.size(); ++i)
                st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
    }
    t query(int l, int r) const {
        assert(l <= r);
        int k = lg[r - l + 1];
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};

```

### 7.5 Lazy Tag

```

// range add, range sum; public ranges are [l, r]
struct lazy_sum {
    int n;
    vector<long long> sum, tag;
    lazy_sum(int n) : n(n), sum(4 * max(1, n)), tag(sum.size()) {
        assert(n > 0);
    }
    void apply(int o, int l, int r, long long v) {
        sum[o] += v * (r - l);
        tag[o] += v;
    }
    void push(int o, int l, int r) {
        if (!tag[o]) return;

```

```

        int m = (l + r) / 2;
        apply(o * 2, l, m, tag[o]);
        apply(o * 2 + 1, m, r, tag[o]);
        tag[o] = 0;
    }
    void add(int ql, int qr, long long v, int o, int l, int r) {
        if (ql <= l && r <= qr) return apply(o, l, r, v);
        push(o, l, r);
        int m = (l + r) / 2;
        if (ql < m) add(ql, qr, v, o * 2, l, m);
        if (m < qr) add(ql, qr, v, o * 2 + 1, m, r);
        sum[o] = sum[o * 2] + sum[o * 2 + 1];
    }
    long long query(int ql, int qr, int o, int l, int r) {
        if (ql <= l && r <= qr) return sum[o];
        push(o, l, r);
        int m = (l + r) / 2;
        long long ans = 0;
        if (ql < m) ans += query(ql, qr, o * 2, l, m);
        if (m < qr) ans += query(ql, qr, o * 2 + 1, m, r);
        return ans;
    }
    void add(int l, int r, long long v) {
        if (l <= r) add(l, r + 1, v, 1, 0, n);
    }
    long long query(int l, int r) {
        return l <= r ? query(l, r + 1, 1, 0, n) : 0;
    }
};

```

### 7.6 BIT 樹狀樹組

```

// public indices are 0-based
struct bit {
    vector<long long> t;
    bit(int n) : t(n + 1) {}

    void add(int p, long long x) {
        for (++p; p < (int)t.size(); p += p & -p) t[p] += x;
    }
    long long prefix(int r) const { // [0, r]
        long long ans = 0;
        for (++r; r > 0; r -= r & -r) ans += t[r];
        return ans;
    }
    long long sum(int l, int r) const { // [l, r]
        return l > r ? 0 : prefix(r) - (l ? prefix(l - 1) : 0);
    }
};

```

### 7.7 並查集 union find

```

struct dsu {
    vector<int> parent, size;
    dsu(int n) : parent(n), size(n, 1) { iota(parent.begin(), parent.end(), 0); }
    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
};

```

## 7.8 Treap

```
// implicit/key treap primitives; node 0 means null
struct treap {
    struct node {
        int value = 0, priority = 0, size = 0, left = 0, right = 0;
    };
    struct split_result { int left, right; };
    vector<node> t{node{}};
    mt19937 rng{uint32_t(chrono::steady_clock::now().
        time_since_epoch().count())};

    int make_node(int value) {
        t.push_back({value, int(rng()), 1, 0, 0});
        return t.size() - 1;
    }
    int size(int u) const { return u ? t[u].size : 0; }
    void pull(int u) { t[u].size = 1 + size(t[u].left)
        + size(t[u].right); }

    split_result split_size(int u, int k) {
        if (!u) return {0, 0};
        if (size(t[u].left) >= k) {
            auto s = split_size(t[u].left, k);
            t[u].left = s.right;
            pull(u);
            return {s.left, u};
        }
        auto s = split_size(t[u].right, k - size(t[u].left) - 1);
        t[u].right = s.left;
        pull(u);
        return {u, s.right};
    }
    split_result split_value(int u, int value) {
        if (!u) return {0, 0};
        if (t[u].value <= value) {
            auto s = split_value(t[u].right, value);
            t[u].right = s.left;
            pull(u);
            return {u, s.right};
        }
        auto s = split_value(t[u].left, value);
        t[u].left = s.right;
        pull(u);
        return {s.left, u};
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (t[a].priority < t[b].priority) {
            t[a].right = merge(t[a].right, b);
            pull(a);
            return a;
        }
        t[b].left = merge(a, t[b].left);
        pull(b);
        return b;
    }
};
```

## 7.9 2D BIT

```
// 1-based rectangle add and rectangle sum; memory is
// 32*n*m bytes
struct bit2d {
    using ll = long long;
    int n, m;
    array<vector<vector<ll>>, 4> t;

    bit2d(int n, int m) : n(n), m(m) {
        for (auto &a : t) a.assign(n + 1, vector<ll>(m + 1));
    }

    void add_one(int x, int y, ll v) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) {
                t[0][i][j] += v;
            }
    }
};
```

```
        t[1][i][j] += v * (x - 1);
        t[2][i][j] += v * (y - 1);
        t[3][i][j] += v * (x - 1) * (y - 1);
    }
}

ll get(int k, int x, int y) const {
    ll ans = 0;
    for (int i = x; i > 0; i -= i & -i)
        for (int j = y; j > 0; j -= j & -j) ans +=
            t[k][i][j];
    return ans;
}

void add(int x1, int y1, int x2, int y2, ll v) {
    assert(1 <= x1 && x1 <= x2 && x2 <= n);
    assert(1 <= y1 && y1 <= y2 && y2 <= m);
    add_one(x1, y1, v);
    if (y2 < m) add_one(x1, y2 + 1, -v);
    if (x2 < n) add_one(x2 + 1, y1, -v);
    if (x2 < n && y2 < m) add_one(x2 + 1, y2 + 1, v);
}

ll prefix(int x, int y) const {
    if (x <= 0 || y <= 0) return 0;
    return get(0, x, y) * x * y - get(1, x, y) * y
        - get(2, x, y) * x + get(3, x, y);
}

ll sum(int x1, int y1, int x2, int y2) const {
    return prefix(x2, y2) - prefix(x1 - 1, y2) -
        prefix(x2, y1 - 1) + prefix(x1 - 1, y1 - 1);
}
};
```

## 8 String

### 8.1 KMP

```
vector<int> prefix_function(const string &s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp(const string &text, const string &
    pattern) {
    if (pattern.empty()) return {};
    vector<int> pi = prefix_function(pattern), pos;
    for (int i = 0, j = 0; i < (int)text.size(); ++i) {
        while (j && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == (int)pattern.size()) {
            pos.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return pos;
}
```

### 8.2 smallest rotation

```
string smallest_rotation(string s) {
    int n = s.size();
    if (!n) return s;
    s += s;
    int i = 0, j = 1;
    while (i < n && j < n) {
```

```

    int k = 0;
    while (k < n && s[i + k] == s[j + k]) ++k;
    if (k == n) break;
    if (s[i + k] < s[j + k]) j += k + 1;
    else i += k + 1;
    if (i == j) ++j;
}
return s.substr(min(i, j), n);
}

```

### 8.3 Suffix Array

```

struct suffix_data {
    vector<int> sa, rank, lcp;
};

suffix_data suffix_array(const string &s) {
    int n = s.size();
    if (!n) return {};
    vector<int> sa(n), rank(n), next(n), temp(n), count(
        max(n, 256));

    for (unsigned char c : s) ++count[c];
    partial_sum(count.begin(), count.end(), count.begin(
        ));
    for (int i = n - 1; i >= 0; --i) sa[--count[(
        unsigned char)s[i]]] = i;
    int classes = 0;
    for (int i = 0; i < n; ++i) {
        if (i && s[sa[i]] != s[sa[i - 1]]) ++classes;
        rank[sa[i]] = classes;
    }
    ++classes;

    for (int len = 1; len < n && classes < n; len *= 2)
    {
        int p = 0;
        for (int i = max(0, n - len); i < n; ++i) temp[
            p++] = i;
        for (int x : sa) if (x >= len) temp[p++] = x -
            len;

        fill(count.begin(), count.begin() + classes, 0)
            ;
        for (int x : temp) ++count[rank[x]];
        partial_sum(count.begin(), count.begin() +
            classes, count.begin());
        for (int i = n - 1; i >= 0; --i)
            sa[--count[rank[temp[i]]]] = temp[i];

        next[sa[0]] = 0;
        int new_classes = 1;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            int ar = a + len < n ? rank[a + len] : -1;
            int br = b + len < n ? rank[b + len] : -1;
            if (rank[a] != rank[b] || ar != br) ++
                new_classes;
            next[b] = new_classes - 1;
        }
        rank.swap(next);
        classes = new_classes;
    }

    vector<int> lcp(n);
    for (int i = 0, h = 0; i < n; ++i) {
        if (!rank[i]) continue;
        int j = sa[rank[i] - 1];
        while (i + h < n && j + h < n && s[i + h] == s[
            j + h]) ++h;
        lcp[rank[i]] = h;
        if (h) --h;
    }
    return {sa, rank, lcp};
}

```

### 8.4 Z-value

```

vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}

// radius of odd palindrome centered at i, including
// the center
vector<int> manacher_odd(const string &s) {
    int n = s.size();
    vector<int> radius(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 1 : min(radius[l + r - i], r -
            i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[
            i + k]) ++k;
        radius[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    return radius;
}

```

### 8.5 旋轉哈希

```

using u64 = uint64_t;
using u128 = __uint128_t;
const u64 mod = (1ULL << 61) - 1;

u64 mod_add(u64 a, u64 b) {
    u64 x = a + b;
    return x >= mod ? x - mod : x;
}

u64 mod_mul(u64 a, u64 b) { return u128(a) * b % mod; }

struct rolling_hash {
    u64 base;
    vector<u64> hash, power;

    rolling_hash(const string &s, u64 base = 911382323)
        : base(base) {
        assert(1 < base && base < mod);
        hash.resize(s.size() + 1);
        power.assign(s.size() + 1, 1);
        for (int i = 0; i < (int)s.size(); ++i) {
            power[i + 1] = mod_mul(power[i], base);
            hash[i + 1] = mod_add(mod_mul(hash[i], base
                ), (unsigned char)s[i] + 1);
        }
    }

    u64 get(int l, int r) const { // [l, r]
        u64 cut = mod_mul(hash[l], power[r - l + 1]);
        return hash[r + 1] >= cut ? hash[r + 1] - cut :
            hash[r + 1] + mod - cut;
    }
};

```

### 8.6 後綴自動機

```

struct sam {
    struct state {
        int len = 0, link = -1;
        array<int, 26> next{};
    };
    vector<state> st{state{}};
    int last = 0;

    void add(char ch) {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        int cur = st.size(), p = last;
        st.push_back({});
        st[cur].len = st[last].len + 1;
    }
};

```

```

while (p != -1 && !st[p].next[c])
    st[p].next[c] = cur, p = st[p].link;
if (p == -1) st[cur].link = 0;
else {
    int q = st[p].next[c];
    if (st[p].len + 1 == st[q].len) st[cur].
        link = q;
    else {
        int clone = st.size();
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        while (p != -1 && st[p].next[c] == q)
            st[p].next[c] = clone, p = st[p].
                link;
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}
};

```

## 9 Others

### 9.1 矩陣樹定理

```

using ll = long long;

ll mod_power(ll a, ll n, ll mod) {
    ll ans = 1;
    for (; n; n /= 2, a = (__int128)a * a % mod)
        if (n & 1) ans = (__int128)ans * a % mod;
    return ans;
}

// undirected multigraph; mod must be prime
ll spanning_trees(int n, const vector<pair<int, int>> &
    edges, ll mod) {
    if (n == 1) return 1;
    vector<vector<ll>> a(n - 1, vector<ll>(n - 1));
    for (auto e : edges) {
        int u = e.first, v = e.second;
        if (u == v) continue;
        if (u < n - 1) a[u][u] = (a[u][u] + 1) % mod;
        if (v < n - 1) a[v][v] = (a[v][v] + 1) % mod;
        if (u < n - 1 && v < n - 1) {
            a[u][v] = (a[u][v] + mod - 1) % mod;
            a[v][u] = (a[v][u] + mod - 1) % mod;
        }
    }
    ll det = 1;
    for (int col = 0; col < n - 1; ++col) {
        int pivot = col;
        while (pivot < n - 1 && !a[pivot][col]) ++pivot;
        if (pivot == n - 1) return 0;
        if (pivot != col) swap(a[pivot], a[col]), det =
            mod - det;
        det = (__int128)det * a[col][col] % mod;
        ll inv = mod_power(a[col][col], mod - 2, mod);
        for (int row = col + 1; row < n - 1; ++row) {
            ll mul = (__int128)a[row][col] * inv % mod;
            for (int j = col; j < n - 1; ++j)
                a[row][j] = (a[row][j] - (__int128)mul
                    * a[col][j] % mod + mod) % mod;
        }
    }
    return det;
}

```

### 9.2 1D/1D dp 優化

```

using ld = long double;

ld int_power(ld x, int p) {
    ld ans = 1;
    for (; p; p /= 2, x *= x) if (p & 1) ans *= x;
}

```

```

return ans;
}

// original line-breaking transition; returns dp[n]
ld line_break_dp(const vector<int> &length, int limit,
    int power) {
    struct interval { int l, r, pos; };
    int n = length.size();
    vector<long long> prefix(n + 1);
    vector<ld> dp(n + 1);
    for (int i = 1; i <= n; ++i) prefix[i] = prefix[i -
        1] + length[i - 1];
    auto cost = [&](int i, int j) {
        ld extra = fabsl(prefix[i] - prefix[j] + i - j
            - 1 - limit);
        return dp[j] + int_power(extra, power);
    };

    vector<interval> stack(n + 2);
    int top = 0, bottom = 0;
    stack[0] = {1, n + 1, 0};
    for (int i = 1; i <= n; ++i) {
        while (i >= stack[bottom].r) ++bottom;
        dp[i] = cost(i, stack[bottom].pos);
        while (top > bottom && i < stack[top].l &&
            cost(stack[top].l, i) < cost(stack[top].
                l, stack[top].pos)) {
            stack[top - 1].r = stack[top].r;
            --top;
        }
        int l = stack[top].l, r = stack[top].r, old =
            stack[top].pos;
        while (l < r) {
            int m = (l + r) / 2;
            if (cost(m, i) < cost(m, old)) r = m;
            else l = m + 1;
        }
        if (l < stack[top].r) {
            int old_r = stack[top].r;
            stack[top].r = l;
            stack[++top] = {l, old_r, i};
        }
    }
    return dp[n];
}

```

### 9.3 Theorem - DP optimization

Monotonicity & 1D/1D DP & 2D/1D DP

Definition xD/yD

1D/1D  $DP[j] = \min_{0 \leq i < j} \{ DP[i] + w(i, j) \}$ ;  $DP[0] = k$   
 2D/1D  $DP[i][j] = \min_{i < k \leq j} \{ DP[i][k - 1] + DP[k][j] \}$   
 $+ w(i, j)$ ;  $DP[i][i] = 0$

Monotonicity

|   | c         | d         |
|---|-----------|-----------|
| a | $w(a, c)$ | $w(a, d)$ |
| b | $w(b, c)$ | $w(b, d)$ |

Monge Condition

Concave (凹四邊形不等式):  $w(a, c) + w(b, d) \geq w(a, d) + w(b, c)$   
 Convex (凸四邊形不等式):  $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$

Totally Monotone

Concave (凹單調):  $w(a, c) \leq w(b, d) \rightarrow w(a, d) \leq w(b, c)$   
 Convex (凸單調):  $w(a, c) \geq w(b, d) \rightarrow w(a, d) \geq w(b, c)$

1D/1D DP  $O(n^2) \rightarrow O(n \lg n)$

\*\*CONSIDER THE TRANSITION POINT\*\*

Solve 1D/1D Concave by Stack

Solve 1D/1D Convex by Deque

2D/1D Convex DP (Totally Monotone)  $O(n^3) \rightarrow O(n^2)$

$$h(i, j - 1) \leq h(i, j) \leq h(i + 1, j)$$

## 9.4 Stable Marriage

```
// preference[a][rank] = b; order[b][a] = rank
vector<int> stable_marriage(const vector<vector<int>> &
                           preference,
                           const vector<vector<int>> &
                           order) {
    int n = preference.size();
    vector<int> next(n), partner(n, -1);
    queue<int> free;
    for (int a = 0; a < n; ++a) free.push(a);
    while (!free.empty()) {
        int a = free.front();
        free.pop();
        int b = preference[a][next[a]++];
        if (partner[b] == -1) partner[b] = a;
        else if (order[b][a] < order[b][partner[b]])
            free.push(partner[b]), partner[b] = a;
        else free.push(a);
    }
    return partner; // partner[b] = a
}
```

## 9.5 莫隊

```
// all ranges are [l, r]
struct query {
    int l, r, id;
};

void sort_mo(vector<query> &queries, int n) {
    int block = max(1, int(n / sqrt(max<size_t>(1,
        queries.size()))));
    sort(queries.begin(), queries.end(), [&](query a,
        query b) {
        int x = a.l / block, y = b.l / block;
        if (x != y) return x < y;
        return x & 1 ? a.r < b.r : a.r > b.r;
    });
}

/*
start with l=0, r=-1 (empty).
move in this order:
while (l > q.l) add(--l); while (r < q.r) add(++r);
while (l < q.l) remove(l++); while (r > q.r) remove(r
--);
*/
```

## 9.6 莫隊帶修改

```
// all ranges are [l, r]; time = number of applied
updates
struct time_query {
    int l, r, time, id;
};

void sort_mo_with_updates(vector<time_query> &queries,
    int n) {
    int block = max(1, int(pow(max(1, n), 2.0 / 3)));
    sort(queries.begin(), queries.end(), [&](time_query
        a, time_query b) {
        int al = a.l / block, bl = b.l / block;
        if (al != bl) return al < bl;
        int ar = a.r / block, br = b.r / block;
        if (ar != br) return al & 1 ? ar < br : ar > br
            ;
        return ar & 1 ? a.time < b.time : a.time > b.
            time;
    });
}

// start with l=0, r=-1, time=0; apply/undo updates
while moving time
```

## 9.7 矩陣乘法

```
using ll = long long;
using matrix = vector<vector<ll>>;

matrix multiply(const matrix &a, const matrix &b, ll
    mod) {
    assert(!a.empty() && !b.empty() && a[0].size() == b
        .size());
    matrix c(a.size(), vector<ll>(b[0].size()));
    for (int i = 0; i < (int)a.size(); ++i)
        for (int k = 0; k < (int)b.size(); ++k)
            for (int j = 0; j < (int)b[0].size(); ++j)
                c[i][j] = (c[i][j] + (__int128)a[i][k]
                    * b[k][j]) % mod;
    return c;
}
```

## 9.8 c++ 小抄

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;

using ordered_set = tree<int, null_type, less<int>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;

// s.find_by_order(k): iterator to k-th smallest, 0-
// based
// s.order_of_key(x): number of values strictly smaller
// than x

using pairing_heap = __gnu_pbds::priority_queue<int,
    less<int>, pairing_heap_tag>;
// auto it = q.push(x); q.modify(it, y); q.erase(it); q
// .join(other);

// __builtin_popcount(x), __builtin_popcountll(x)
// __builtin_clz(x), __builtin_ctz(x): x must be
// nonzero
// __builtin_*_overflow(a, b, &answer)

int random_int(int l, int r) {
    static mt19937 rng(chrono::steady_clock::now().
        time_since_epoch().count());
    return uniform_int_distribution<int>(l, r)(rng);
}

// sorted unique:
// a.erase(unique(a.begin(), a.end()), a.end());
```

## 9.9 python 小抄

```
#!/usr/bin/env python3
from collections import deque
import sys

def ints():
    return map(int, input().split())

def all_ints():
    return list(map(int, sys.stdin.buffer.read().split
        ()))

def lcs(a, b):
    dp = [0] * (len(b) + 1)
    for x in a:
        prev = 0
        for j, y in enumerate(b, 1):
            old = dp[j]
            if x == y:
                dp[j] = prev + 1
```

```

        else:
            dp[j] = max(dp[j], dp[j - 1])
            prev = old
    return dp[-1]

# stack: append, pop, stack[-1]
# queue = deque(); append, popleft, queue[0]
# sorted unique values: sorted(set(values))

```

## 9.10 萬年曆

$$h = \left( q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor + 5J \right) \bmod 7$$

h : 星期 (0 = 星期六, 1 = 星期日, 2 = 星期一, ...)  
 q : 日期 (日)  
 m : 月份 (3= 三月, ...; 一、二月改成 13、14, 且年份先減一)  
 K : 調整後年份的後兩位數 (year mod 100)  
 J : 調整後年份的前兩位數 (year ÷ 100)

## 9.11 多重背包

```

using ll = long long;

struct item {
    int weight, count;
    ll value;
};

// dp[w] = maximum value with total weight at most w
vector<ll> bounded_knapsack(int capacity, const vector<
    item> &items) {
    vector<ll> dp(capacity + 1);
    for (item x : items) {
        assert(x.weight >= 0 && x.count >= 0);
        if (!x.weight) {
            if (x.value > 0)
                for (ll &value : dp) value += x.value *
                    x.count;
            continue;
        }
        for (int r = 0; r < x.weight && r <= capacity;
            ++r) {
            deque<pair<ll, int>> q;
            for (int k = 0, w = r; w <= capacity; ++k,
                w += x.weight) {
                ll value = dp[w] - ll(k) * x.value;
                while (!q.empty() && q.back().first <=
                    value) q.pop_back();
                q.push_back({value, k});
                while (q.front().second < k - x.count)
                    q.pop_front();
                dp[w] = q.front().first + ll(k) * x.
                    value;
            }
        }
    }
    return dp;
}

```